Cloud readiness verified. Deployment awaiting account access.

Status: IMPLEMENTATION VERIFIED; DEPLOYMENT BLOCKED AT ACCOUNT-CONSOLE ACCESS

Implementation, local regression and Linux readiness are verified. No application service has been provisioned, account-specific card-free eligibility remains unconfirmed, and no public dashboard URL exists. This is not a successful deployment certificate.

Evidence recorded 2026-11-11T21:58:03.108010+00:00 (UTC).

Approved architecture and cost boundary

Three laptop-hosted Wokwi ESP32 clients publish their unchanged four-slot JSON telemetry to EMQX Serverless over verified TLS. One Render Free FastAPI worker ingests the three existing topics and persists to Neon Free PostgreSQL. Streamlit Community Cloud calls the authenticated HTTPS API and uses the existing in-process saved-model interface. The verified local SQLite path remains available.

Service	Published free-plan boundary
Streamlit Community Cloud	Free frontend;12h inactivity sleep; shared CPU/RAM limits.
Render Hobby + Free	750 instance-hours/workspace/month;15min idle sleep; ephemeral filesystem; no always-on subscriber guarantee.
Neon Free	1GB database/project;100CU-hours/month;5GB outbound/project/month;5min idle compute suspension.
EMQX Serverless	Spend Limit exactly0;1M session-minutes and1GB bidirectional traffic/month; suspend at quota.

Total authorized recurring spend: **INR0**. No cards, payment holds, paid add-ons, upgrades, domains or marketplace subscriptions are authorized. Published free quotas do not establish eligibility for a particular account; the provider console must permit card-free creation before proceeding.

Capacity and continuity limits

A captured543-byte payload, three boards at one message per five seconds, and one subscriber imply roughly1.69GB in30 continuous days before overhead. This exceeds the EMQX1GB free traffic quota. Use bounded academic demonstrations and monitor quotas. Neon100CU-hours supports400 active hours at0.25CU, not continuous operation for a full month. Render sleep takes the MQTT subscriber offline; queued redelivery depends on broker/session limits.

References: render.com/docs/free; neon.com/docs/introduction/plans; docs.emqx.com/en/cloud/latest/deployments/spend_limit.html; docs.streamlit.io/deploy/streamlit-community-cloud/manage-your-app. Terms audited November2026; recheck console eligibility at creation.

Implementation and preservation

PostgreSQL adapter and historical continuity

backend/postgres.py reuses the unchanged backend.db persistence methods through a narrow SQL transport adapter. It binds values, maps generated IDs, uses a transaction-scoped advisory lock to serialize writers, and preserves fingerprints, duplicate detection, sequence/session ordering, history-only records and current-slot pointers. History count/rows use a consistent transaction snapshot. SQL errors are sanitized into the existing retry/503 path; MQTT acknowledgement still occurs after a committed write. PostgreSQL requires verify-full TLS with the system certificate store.

The dedicated smartparking schema contains slots, telemetry_messages, occupancy_observations, slot_current and version metadata. IDs use BIGSERIAL; unsigned device sequence/uptime fields use BIGINT. Existing slot, state, distance, identity and duplicate constraints remain. Unknown schemas are refused, not migrated or reset. Default local execution does not import psycopg or change its SQLite database.

A one-time copy tool opens recognized SQLite read-only, preserves original IDs/timestamps/fingerprints, checks source hashes and refuses nonempty destinations. A driver-free SQL export supports the Neon SQL editor without installing a laptop driver. The direct-driver path additionally compares exact copied rows before commit. The SQL-editor path guards empty destinations and validates counts; real PostgreSQL tests exercise both paths. Actual cloud copying has not occurred. Never commit the runtime SQL export or database.

MQTT TLS, API authentication and firmware

Backend MQTT uses a certificate- and hostname-verifying SSL context with minimum TLS1.2. Cloud mode requires PostgreSQL, MQTT TLS/authentication and an API token of at least32 characters. Configured tokens protect API routes and documentation; only minimal /livez is public. Authentication is constant-time and responses disable caching. Secrets are omitted from settings repr and belong in provider-managed configuration, never Git or chat.

Three opt-in *_cloud firmware targets use WiFiClientSecure and a public trusted CA; missing private identity/CA/port settings stop startup instead of falling back to plaintext or insecure verification. Local targets and sensor/GPIO/classification logic are retained. Cloud library settings reuse existing MQTT caches. A helper can prepare ignored simulation folders with byte-identical verified diagrams; it refuses overwrite and leaves local workspaces intact.

Frontend and model interface

API_BASE_URL remains configurable. The cloud frontend uses PREDICTION_ADAPTER=in_process, repository-relative artifact paths and the unchanged Phase4 bundle. Existing historical graphs, model metrics and synthetic retrospective features are preserved. Temporary502/503/504 failures explain possible free-tier wake-up delays. Neither feature pipeline nor separate Persistence-only freshness validation was weakened.

Preservation: original SQLite is401,408 bytes; all protected source/data/model paths and every pre-existing PDF match the baseline.

SQLite SHA-256: 1811930e8212c9bfd7df2d72533da593f3fb45ab2782f35f4dd0f4b6a5dcabe0

Frozen model SHA-256: cfc2adc80cc949e3c097530bca500079b857eee8b931be1e6d7de522f50335e1

Verification evidence and results

Verification	Observed result / evidence boundary
Local Windows suites	PASS: 147 executed tests; 5 real-PostgreSQL tests explicitly skipped locally. Full output archived.
Real PostgreSQL/Linux	PASS: latest GitHub readiness run completed successfully, including PostgreSQL persistence/import/concurrency tests and all backend/dashboard/ML test steps.
Saved model on Linux	PASS: actual unchanged-bundle reload and labelled synthetic retrospective prediction through the existing in-process interface. Not a live forecast.
Firmware	PASS in Linux: all3 local and3 TLS ESP32 targets compiled;3 C++ host regression programs executed. This is not Wokwi cloud runtime verification.
Cloud simulation helper	PASS with isolated filesystem fixtures for all3 boards: exact diagram copy, correct target paths and refusal to overwrite. No actual simulators launched.
Local API/dashboard	PASS: existing dashboard8501/8503 HTTP health and actual API8000 endpoint remain available. Forecast unavailable with12 stale blockers and null values.
Provider deployment	BLOCKED: supported browser host unavailable; no account-console access, service creation, public URL or cloud telemetry evidence.

Successful latest CI source: 0980b786625b6de6fa22415d61b8f9082bbfd0dc

CI run: https://github.com/vighnesh-6/iot-based-smart-parking-system/actions/runs/43106716753

Public GitHub run/job/step conclusions are archived. Raw authenticated Linux logs were not retrieved. The152-test inventory (62backend,45dashboard,42ML,3circuit) is derived from the verified source/local discovery; it is not presented as independently archived Linux stdout. The five PostgreSQL tests are enabled by the disposable CI database configuration. Real broker delivery to EMQX/Neon has not been observed.

Failures investigated and corrected

Run43104923886 failed before testing because ubuntu-latest moved to Ubuntu26.04, which lacked the exact Python3.12.10 build. The runner is now pinned to Ubuntu24.04. Run43105710451 failed dependency installation: official PyPI metadata establishes that PlatformIO requires uvicorn<0.35, conflicting with application uvicorn==0.54.0. PlatformIO now uses a separate hosted environment. Subsequent runs43106498231 and43106716753 succeeded.

Automatic approval review rejected a proposed existing-Git-credential read for private CI logs before execution. No credential was retrieved, stored or exposed; no retry/bypass was attempted. Diagnosis used public official package metadata and public check annotations. Laptop environments, Windows Application Control and security settings were not modified.

Scientific limitations

Every current local observation is stale. Persistence-only inference requires genuine known observations for all12 slots, with both receipt and measurement timestamps fresh within at most30 seconds. Its rule remains occupied(t+15)=occupied(t); stale sequential board runs cannot be combined. The full37-feature path independently requires10,081 complete minute rows. Existing model metrics describe synthetic data only. Neither Linux success nor cloud hosting establishes real-world predictive accuracy.

Deployment gate and continuation

Current blocker and required account step

Computer Use inventory returned browsers=[] and "Browsers: Error: The browser access host is unavailable". No provider console could be inspected or operated through that supported interface. The main Codex/VS Code window was untouched. A console handoff was requested so the operator can use their existing browser without installing a bridge, entering secrets in chat or weakening policy. No provider payment requirement has yet been observed; account-specific card-free eligibility is unconfirmed.

Exact continuation order

1. Operator signs into official provider consoles and confirms Free/no-card eligibility; stop at any payment prompt.
2. Create dedicated Neon Free storage and EMQX Serverless with spend limit0 and authenticated topic permissions.
3. Deploy Render from render.yaml, configure private DATABASE_URL, MQTT identity/TLS and API token, then verify protected health.
4. Copy existing history into the empty initialized schema using the guarded SQL export; verify counts/content and preserve the local source.
5. Deploy dashboard/app.py on Streamlit with API_BASE_URL, the matching private token and in-process model execution.
6. Configure/build private cloud firmware using existing cached tools and run one additional simulator window at a time. Capture actual production messages and exact PostgreSQL observations for all boards; classify evidence as sequential when appropriate.
7. Verify dashboard cards, filters, gaps, refresh, cold starts, API failures and unchanged freshness gates; record the actual public dashboard URL. Only then mark deployment complete.

Configuration and reproduction

Backend environment: DEPLOYMENT_MODE=cloud; DATABASE_URL=private verified-TLS Neon URL; MQTT_TLS=true; MQTT_PORT=8883; assigned MQTT_HOST; private MQTT_USERNAME/MQTT_PASSWORD; unique cloud MQTT_CLIENT_ID; private API_BEARER_TOKEN; STALE_SECONDS=30. Frontend root-level secrets: HTTPS API_BASE_URL, matching token and PREDICTION_ADAPTER=in_process. Use Python3.12 and the pinned hosted dependency manifests. Never use a Windows ML_PYTHON path in Linux.

Driver-free history export: python -B -m backend.tools.import_sqlite_history --source <original external SQLite path> --export-sql tmp/phase7_history_import.sql. Source file remains read-only; export refuses overwrite. Run only against an empty dedicated cloud destination. Detailed commands, account gates and provider links are in reports/phase7_deployment_runbook.md.

Final acceptance item	Status
Cloud implementation and regression readiness	PASS
Frozen artifacts and original SQLite preservation	PASS
Laptop environments/security unchanged; zero spending	PASS

Final acceptance item	Status
Account-specific free/card-free eligibility	PENDING
Four-service deployment and public URL	PENDING
Actual cloud MQTT-to-PostgreSQL/dashboard delivery	UNVERIFIED
Genuine fresh twelve-slot live forecast	UNVERIFIED

Laptop dependency: Wokwi telemetry stops when the laptop/simulator stops or pauses. Cloud history remains history; stale/missing states must stay explicit. This architecture is a bounded free academic demonstration, not reliable24/7 monitoring. No cloud deployment or live prediction is claimed.